

Guida Bash – File Descriptor, Scrittura su File e Cicli `for` (versione da esame)

Guida **mirata da esame** sull'uso corretto di:

- **File Descriptor (FD)**
- **scrittura su file** (stdout, stderr, append, newline)
- **cicli** `` (nomi file, argomenti, pattern reali)

Obiettivo: sotto pressione, sapere **cosa usare, quando e perché**, senza introdurre costrutti fuori programma.

1. File Descriptor: modello operativo minimo

Ogni processo Bash ha stream numerati:

FD	Significato
0	stdin
1	stdout
2	stderr
3+	file aperti manualmente

In esame:

- FD 0,1,2 → sempre presenti
- $FD \geq 3$ → solo se usi `exec`

2. `exec`: aprire e gestire File Descriptor

2.1 Apertura in lettura

```
exec {FD}< file.txt
```

- Bash assegna un FD libero (es. 3)
- il numero è salvato in `FD`

Controllo corretto:

```
exec {FD}< file.txt || exit 1
```

2.2 Lettura da FD

```
read -u ${FD} RIGA
```

Pattern standard:

```
exec {FD}< file.txt
while read -u ${FD} RIGA ; do
    echo "${RIGA}"
done
exec {FD}>&-
```

⚠ errore tipico: dimenticare `-u ${FD}` → leggi da stdin.

2.3 File senza newline finale (caso d'esame)

Problema:

- l'ultima riga non termina con `\n`

Soluzione corretta:

```
while read -u ${FD} RIGA || [[ "${RIGA}" != "" ]] ; do
    echo "${RIGA}"
done
```

Questa forma va **memorizzata**.

3. Scrittura su file con FD

3.1 Sovrascrittura

```
exec {FD}> output.txt
```

- il file viene azzerato

3.2 Append

```
exec {FD}>> output.txt
```

- il contenuto precedente resta

3.3 Scrivere su FD

```
echo "test" 1>&{FD}
```

Differenza concettuale:

- `echo ... > file` → apre/chiude ogni volta
- `exec {FD}> file` → file aperto una sola volta

4. Chiusura dei FD

Ogni FD aperto manualmente va chiuso:

```
exec {FD}>&-
```

⚠️ errori da esame:

- FD non chiuso
- uso di FD dopo la chiusura

5. Redirezioni standard VS FD

5.1 Redirezioni semplici (preferite)

```
while read R ; do  
    echo "$R"  
done < in.txt > out.txt
```

✓ più leggibile ✓ meno rischio

5.2 Quando usare `exec`

Caso	<code>exec</code>
una sola lettura	✗
una sola scrittura	✗
più file aperti	✓
richiesta esplicita	✓

6. `echo` e problemi con le newline

6.1 `echo` aggiunge newline

```
echo "ciao" # ciao\n
```

Effetti:

- `wc -c` conta anche il newline

6.2 Senza newline

Se consentito:

```
echo -n "ciao"
```

Usare solo se richiesto.

7. `read -N` e `read -n`

7.1 `read -N 1`

- legge anche `\n`
- utile per conteggi precisi

7.2 `read -n 1`

- si ferma prima del newline

Errore tipico: conteggi sbagliati di 1.

8. Scrittura su stderr

```
echo "errore" 1>&2
```

Separare output ed errori è spesso valutato bene.

9. Cicli `for` (fondamentali da esame)

9.1 `for` su argomenti

```
for A in "$@" ; do
    echo "$A"
done
```

✓ mantiene i confini

Errore tipico:

```
for A in $@ ; do    # SBAGLIATO
```

9.2 `for` su nomi di file

```
for F in DIR/* ; do
    echo "$F"
done
```

✓ gestisce spazi ✓ niente `ls`

9.3 Aggiungere estensione ai file

```
for F in DIR/* ; do
    cp "$F" "$F.txt"
done
```

9.4 `for` con pattern speciali

```
for F in BUTTAMI/* ; do
    echo "$F"
done
```

Usare sempre virgolette.

9.5 `for` numerico (se ammesso)

```
for (( i=0; i<10; i++ )) ; do
    echo "$i"
done
```

Usare solo se presente nei PDF.

10. `for` VS `while read`

Situazione	Usa
leggere righe	<code>while read</code>
argomenti	<code>for</code>
file	<code>for DIR/*</code>

11. Pattern d'esame ricorrenti

Pattern A — copia file con modifica nome

- `for`
- quoting

Pattern B — filtro + scrittura

- `while read`
- redirectione o FD

Pattern C — conteggio caratteri

- `read -N`

Pattern D — errori su stderr

- `echo 1>&2`
-

12. Decision tree rapido

- sto leggendo righe? → `while read`
 - sto iterando file? → `for DIR/*`
 - devo scrivere su file? → `>` o `>>`
 - devo gestire FD? → `exec`
 - newline importante? → attenzione `echo`
-

Domanda critica

Quando puoi risolvere un esercizio sia con `for` sia con `while read`, quale scegli e in base a quale criterio stai davvero ottimizzando: **chiarezza**, **robustezza** o **abitudine**?